PS1

Written By: Matthew Averill

1) Activation Functions

(A) Non-linear activation functions allow the neural network to describe non-linear relations between input and the output. If a linear activation function was used at each hidden unit, the mapping from input to output would be restricted to be linear.

(B) Two example activation functions

1. ReLU

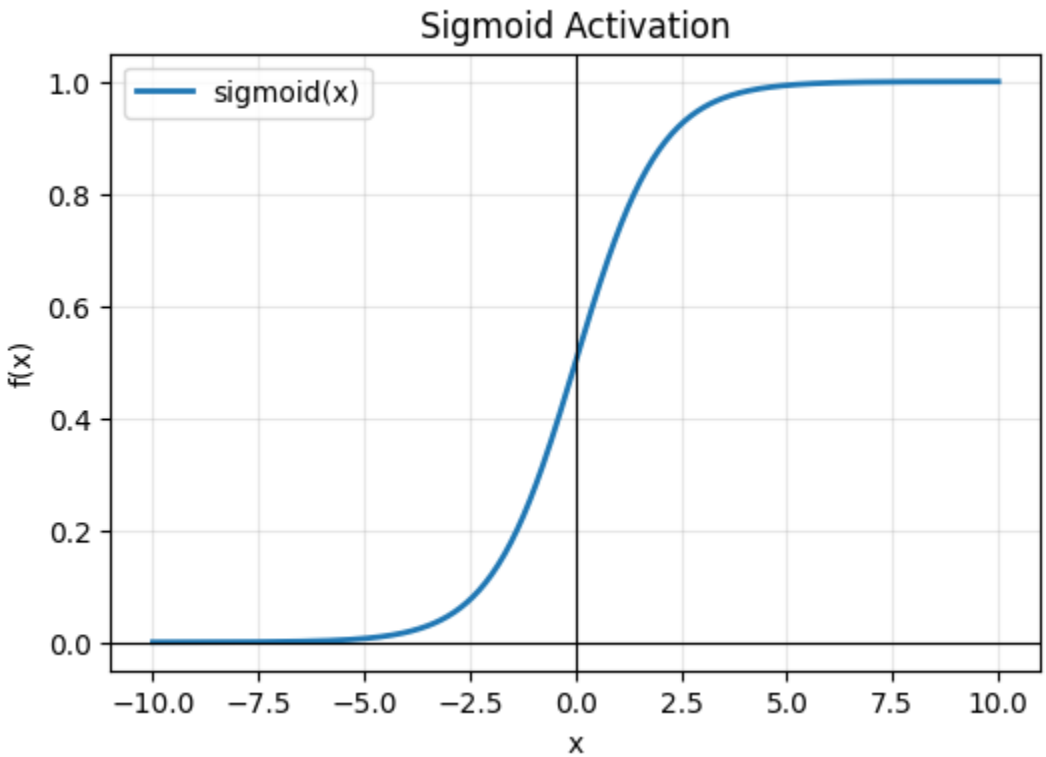
$$\mathrm{a}[z] = \mathrm{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

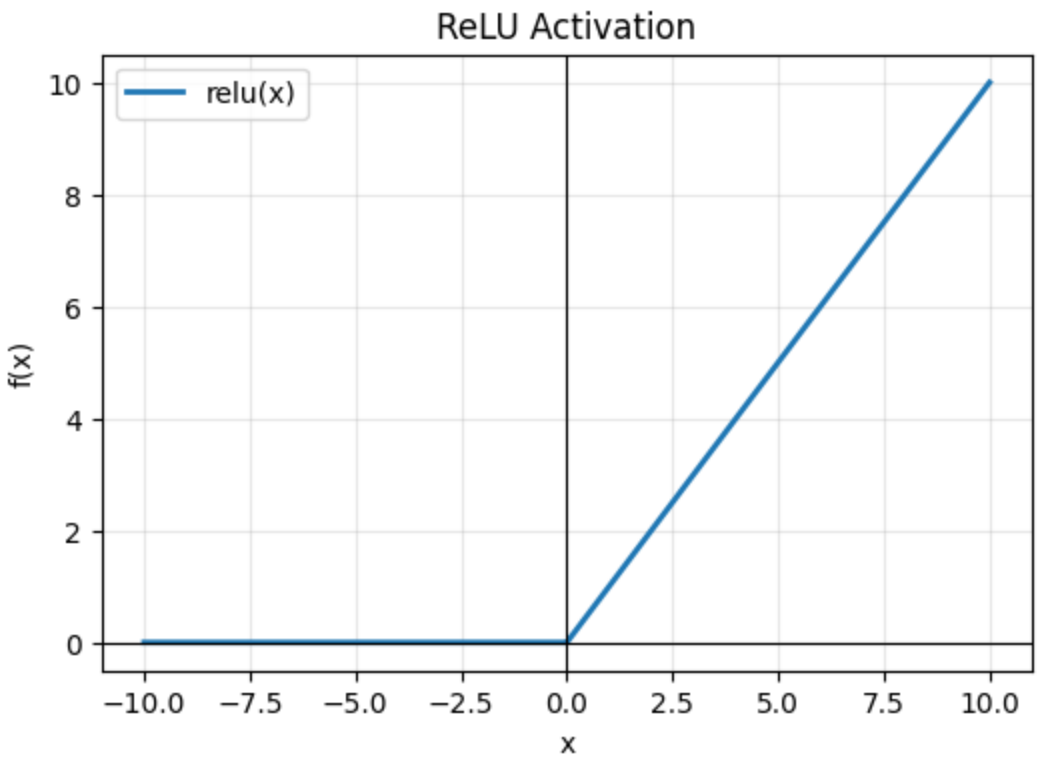
2. Logistic Sigmoid

$$\mathrm{sig}[z] = \frac{1}{1 + \exp[-z]}$$

In [5]:
```
import os
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

os.makedirs('images', exist_ok=True)

x = np.linspace(-10, 10, 400)
y_sigmoid = sigmoid(x)
y_relu = relu(x)
```

```python
# Sigmoid
plt.figure(figsize=(6, 4))
plt.plot(x, y_sigmoid, label='sigmoid(x)', linewidth=2)
plt.axhline(0, color='black', linewidth=0.8)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Sigmoid Activation')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.savefig('images/sigmoid_activation.png', dpi=150, bbox_inches='tight')
plt.show()

# ReLU
plt.figure(figsize=(6, 4))
plt.plot(x, y_relu, label='relu(x)', linewidth=2)
plt.axhline(0, color='black', linewidth=0.8)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('ReLU Activation')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.savefig('images/relu_activation.png', dpi=150, bbox_inches='tight')
plt.show()
```

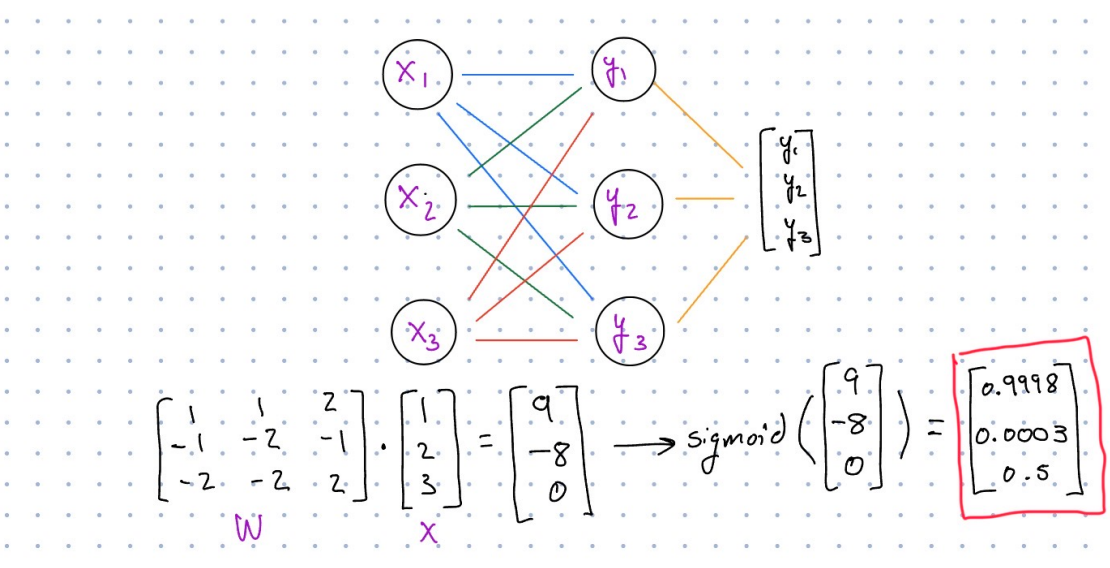




(C) A regression problem returns a continuous number for a variable you are trying to predict as its ouput, while a classification problem returns a category assignment via a probability for each potential class in the form of a vector. For regression I would use a linear activation function that essentially passes the value through to the output. Since the continuous number could be positive or negative, I would not use a ReLU activation function. For a classification problem, I would use the sigmoid activation function because it outputs a value between 0-1 which would represent a probability.

(D)

(i)



(ii)

$$\underset{2\times 3}{\begin{bmatrix} -1 & -1 & 1 \\ 1 & 2 & 1 \end{bmatrix}} \cdot \underset{3\times 1}{\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}} = \underset{2\times 1}{\begin{bmatrix} 0 \\ 8 \end{bmatrix}}$$

$$\text{ReLU}\left(\begin{bmatrix} 0 \\ 8 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 8 \end{bmatrix}$$

$$\underset{2\times 2}{\begin{bmatrix} 1 & -1 \\ 1 & 2 \end{bmatrix}} \cdot \underset{2\times 1}{\begin{bmatrix} 0 \\ 8 \end{bmatrix}} = \underset{2\times 1}{\begin{bmatrix} -8 \\ 16 \end{bmatrix}}$$

$$\text{ReLU}\left(\begin{bmatrix} -8 \\ 16 \end{bmatrix}\right) = \boxed{\begin{bmatrix} 0 \\ 16 \end{bmatrix}}$$

2) Loss Functions

(A)

In [9]:
```
# Load dataset from Hugging Face
from datasets import load_dataset

dataset_name = "lamm-mit/data_sample"    # change this
ds = load_dataset(dataset_name, download_mode="force_redownload")

print(ds)
```

```
print(ds["train"][0])
```

```
Generating train split: 100%|██████████| 400/400 [00:00<00:00, 69124.54 examples/s]
DatasetDict({
    train: Dataset({
        features: ['Unnamed: 0', 'ground truth', 'prediction'],
        num_rows: 400
    })
})
{'Unnamed: 0': 0, 'ground truth': 0.2186085658412798, 'prediction': 0.23017886617092
34}
```

In [27]:

```
import numpy as np


# Calculate MAE
df = ds["train"].to_pandas()
selected = df[["ground truth", "prediction"]].copy()

y_true = selected["ground truth"].astype(float).to_numpy()
y_pred = selected["prediction"].astype(float).to_numpy()

mae = np.mean(np.abs(y_true - y_pred))
print("MAE: " + str(mae))

# Calculate MSE
mse = np.mean((y_true - y_pred) ** 2)
print("MSE: " + str(mse))

# Coefficient of determination for predictions
y_true_series = selected["ground truth"].astype(float)
y_pred_series = selected["prediction"].astype(float)
ss_res = ((y_true_series - y_pred_series) ** 2).sum()
ss_tot = ((y_true_series - y_true_series.mean()) ** 2).sum()
r2_pred = 1 - (ss_res / ss_tot)
print("R^2 : " + str(r2_pred))
```

```
MAE: 0.01832773426808224
MSE: 0.000458758238585838
R^2 : 0.898312268689691
```

(B) MAE looks at the average absolute error of the predictions. If outliers exist within the dataset, MAE will prevent them from dominating the training error. It is better for when the engineer wants the typical absolute error and potentially for explainability since the units are the same as the gorund truth data. MSE looks at the average of squared mistakes. If there are outliers, they will be penalized heavier due to the squared term. This makes MSE ideal for use cases where large errors are much worse than small ones.

3) Beam Mechanics

(A)

Moment of inertia

distributed load

$$EI\frac{\partial^4 z}{\partial x^4} = q(x)$$

Modulus of Elasticity

<u>Boundary Conditions</u>:

$z(0) = 0$

$M(L) = 0$

$z^4(x) = 0$ when $0 < x < L$

$z(0) = 0$; $z'(0) = 0$; $z''(1) = 0$; $z'''(1) = 1$

(B)

$$\int z^4(x)\,dx = \int 0\,dx \longrightarrow z^3(x) = C_1$$

use BC $z^3(1)=1 \therefore C_1 = 1$

$$\int z^3(x)\,dx = \int 1\,dx \longrightarrow z^2(x) = x + C_2$$

use BC $z^2(1)=0 \therefore 0 = 1 + C_2 \longrightarrow C_2 = -1$

$$\int z^2(x)\,dx = \int (x-1)\,dx \longrightarrow z^1(x) = \frac{1}{2}x^2 - x + C_3$$

use BC $z^1(0)=0 \therefore 0 = C_3$

$$\int z^1(x)\,dx = \int \left(\frac{1}{2}x^2 - x\right)dx \longrightarrow z(x) = \frac{1}{6}x^3 - \frac{1}{2}x^2 + C_4$$

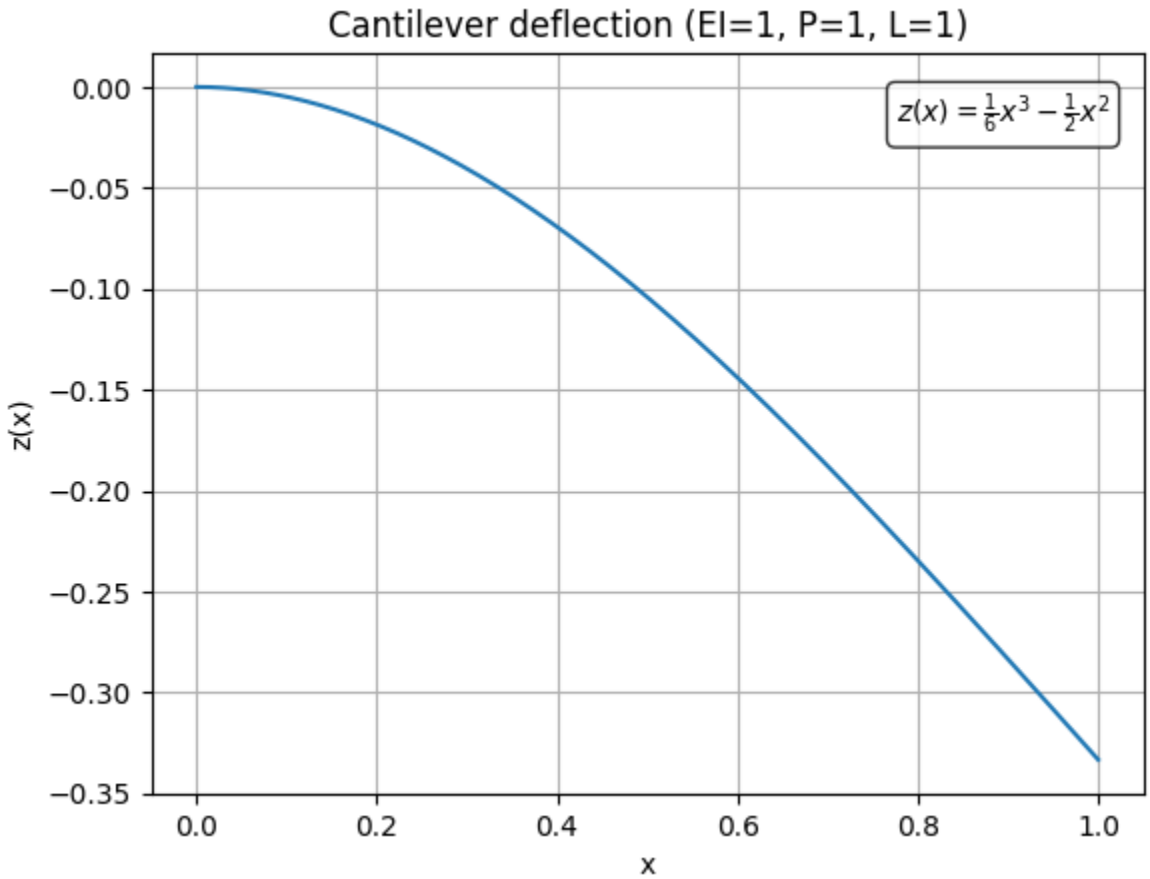
use BC $z(0)=0 \therefore C_4 = 0$

$$\boxed{z(x) = \left[\frac{1}{6}x^3 - \frac{1}{2}x^2 + C_4\right]}$$

(C)

In [33]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 1, 400)
z = (1/6)*x**3 - (1/2)*x**2

plt.figure()
plt.plot(x, z)
plt.xlabel("x")
plt.ylabel("z(x)")
plt.title("Cantilever deflection (EI=1, P=1, L=1)")
plt.grid(True)

eqn = r"$z(x)=\frac{1}{6}x^3-\frac{1}{2}x^2$"
```

```
plt.text(0.75, 0.95, eqn, transform=plt.gca().transAxes,
         verticalalignment="top",
         bbox=dict(boxstyle="round", facecolor="white", alpha=0.8))
plt.show()
```



(D)

$z'(x) = \frac{1}{2}x^2 - x \longrightarrow z'(0) = 0$ ✓

$z''(x) = x - 1 \longrightarrow z''(1) = 0$ ✓

$z'''(x) = 1 \longrightarrow z'''(1) = 1$ ✓

4) Working with Material Databases

(A)

× Include Elements: Al × Bulk Modulus, Voigt: 100 to 300

Material ID	Formula	Crystal System	Space Group Symbol
mp-771	MnAl	Tetragonal	P4/mmm
mp-607111	$AlPt_3$	Tetragonal	P4/mbm
mp-2500	AlCu	Monoclinic	C12/m1
mp-1018166	$AlTc_2$	Tetragonal	I4/mmm
mp-1399	AlAu	Monoclinic	$P12_1/m1$
mp-661	AlN	Hexagonal	$P6_3mc$
mp-10909	$AlRe_2$	Tetragonal	I4/mmm
mp-10904	AlPt	Cubic	$P2_1\bar{3}$
mp-18427	Nb_2Al	Tetragonal	$P4_2/mnm$
mp-1490	$AlAu_4$	Cubic	$P2_1\bar{3}$
mp-1487	AlNi	Cubic	$Pm\bar{3}m$

(B) I would represent each material as a matrix of relevant properties that represent different features.The properties I would choose are outlines in the table headers below.

In [1]:
```
import pandas as pd

cols = [
    "material_id", "formula", "nsites", "space_group", "volume", "structure",
    "elastic_anisotropy", "G_Reuss", "G_VRH", "G_Voigt",
    "K_Reuss", "K_VRH", "K_Voigt", "poisson_ratio",
    "compliance_tensor", "elastic_tensor"
]

df_ml = pd.DataFrame(columns=cols)
df_ml
```

Out[1]:

material_id	formula	nsites	space_group	volume	structure	elastic_anisotropy	G_Reuss

5) Developing and Training a Regression Model Using an MLP

In [8]:
```
# import dataset
```

```
from matminer.datasets import load_dataset
import pandas as pd

# Load dataset
df = load_dataset("brgoch_superhard_training")
print("Available columns:", df.columns.tolist())
```

```
Available columns: ['formula', 'bulk_modulus', 'shear_modulus', 'composition', 'mate
rial_id', 'structure', 'brgoch_feats', 'suspect_value']
```

In [10]:

```
from matminer.featurizers.conversions import StrToComposition
from matminer.featurizers.composition import ElementProperty
from matminer.datasets import load_dataset

target_label = "bulk_modulus"

raw_df = load_dataset("brgoch_superhard_training")
raw_df = raw_df[["formula", target_label]].dropna().copy()

df = StrToComposition().featurize_dataframe(raw_df, "formula")
featurizer = ElementProperty.from_preset("magpie")
df = featurizer.featurize_dataframe(df, "composition", ignore_errors=True)

X = df.drop(columns=["formula", "composition", target_label])
y = df[target_label]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
print("Target column:", target_label)
```

```
StrToComposition: 100%|██████████| 2574/2574 [00:33<00:00, 77.80it/s]
ElementProperty: 100%|██████████| 2574/2574 [02:05<00:00, 20.57it/s]
Feature shape: (2574, 132)
Target shape: (2574,)
Target column: bulk_modulus
```

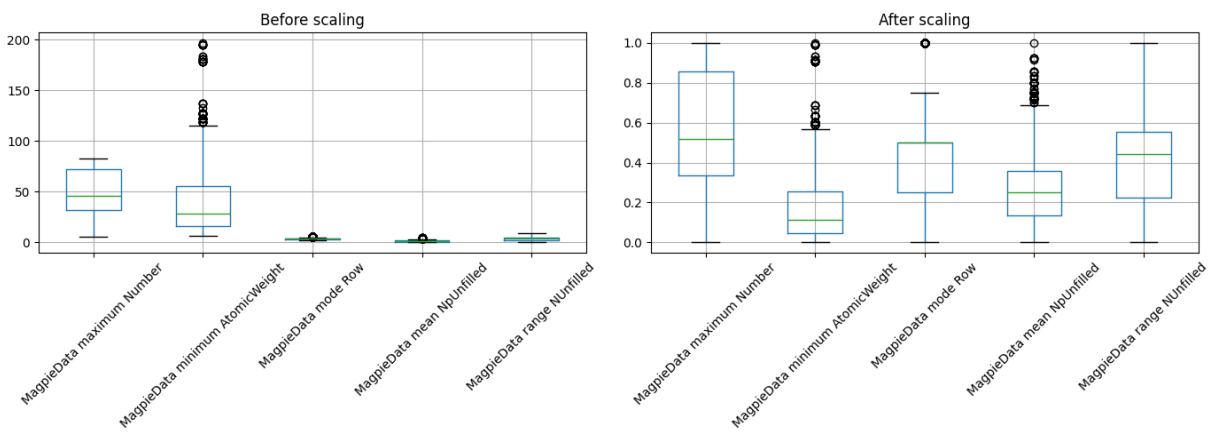
(A) Scale Data

In [21]:

```
from sklearn.preprocessing import MinMaxScaler

X_scaler = MinMaxScaler()
y_scaler = MinMaxScaler()

X_scaled = X_scaler.fit_transform(X)
# MinMaxScaler expects 2D input --> reshape y to be a 2D array with one column
y_scaled = y_scaler.fit_transform(y.to_numpy().reshape(-1, 1)).ravel()

print("X_scaled shape:", X_scaled.shape)
print("y_scaled shape:", y_scaled.shape)
print("X range:", X_scaled.min(), X_scaled.max())
print("y range:", y_scaled.min(), y_scaled.max())
```

```
X_scaled shape: (2574, 132)
y_scaled shape: (2574,)
X range: 0.0 1.0000000000000002
y range: 0.0 1.0
```

In [22]:
```
import matplotlib.pyplot as plt
import pandas as pd

selected_features = X.columns[[1, 12, 35, 87, 104]].tolist()

X_before = X[selected_features]
X_after = pd.DataFrame(X_scaled, columns=X.columns)[selected_features]

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=False)

X_before.boxplot(ax=axes[0])
axes[0].set_title("Before scaling")
axes[0].tick_params(axis="x", rotation=45)

X_after.boxplot(ax=axes[1])
axes[1].set_title("After scaling")
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```



In [35]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torchsummary import summary
from sklearn.model_selection import train_test_split
from tqdm.auto import tqdm

# Split into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_scaled, y_scaled, test_size=0.2

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_val = torch.tensor(X_val, dtype=torch.float32)
y_val = torch.tensor(y_val, dtype=torch.float32).reshape(-1, 1)
```

(B) Train MLP

Define Model

In [36]:
```
# Define MLP model
```

```python
class MLP(nn.Module):
    def __init__(self, input_size):
        super(MLP, self).__init__()
        self.hidden1 = nn.Linear(input_size, 64)
        self.hidden2 = nn.Linear(64, 32)
        self.output = nn.Linear(32, 1)
        self.activation = nn.ReLU()

    def forward(self, x):
        x = self.activation(self.hidden1(x))
        x = self.activation(self.hidden2(x))
        x = self.output(x)
        return x

# Initialize model
model = MLP(input_size=X_train.shape[1])

#View Neural Network architecture
print("\nNeural Network Architecture Summary:")
summary(model, input_size=(1, X_train.shape[1]))
```

```
Neural Network Architecture Summary:
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Linear-1                [-1, 1, 64]           8,512
              ReLU-2                [-1, 1, 64]               0
            Linear-3                [-1, 1, 32]           2,080
              ReLU-4                [-1, 1, 32]               0
            Linear-5                 [-1, 1, 1]              33
================================================================
Total params: 10,625
Trainable params: 10,625
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.04
Estimated Total Size (MB): 0.04
----------------------------------------------------------------
```

Training Loop

In [37]:
```python
# Define loss function and optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training loop with tqdm progress bar
num_epochs = 96
train_losses = []
val_losses = []

for epoch in tqdm(range(num_epochs), desc="Training Progress"):
    model.train()
    optimizer.zero_grad()
```

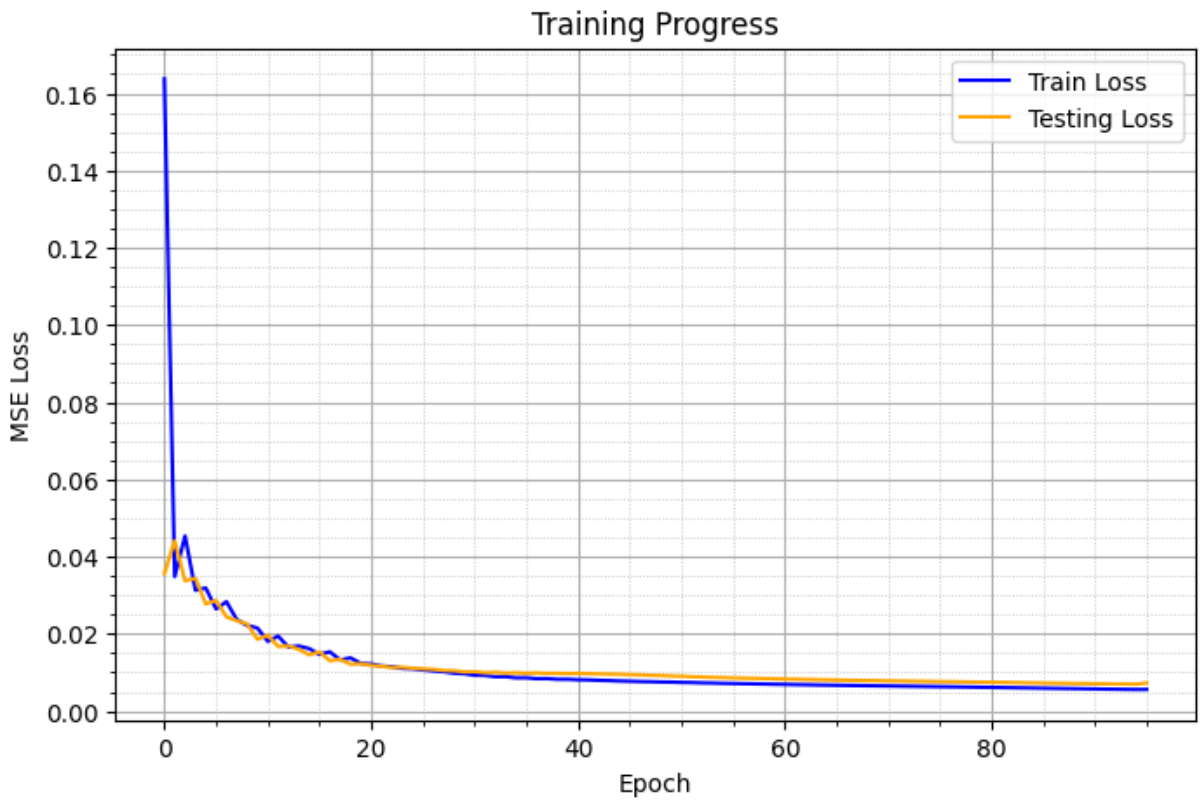
```
    # Forward pass
    y_pred_train = model(X_train)
    loss = criterion(y_pred_train, y_train)

    # Backward pass
    loss.backward()
    optimizer.step()

    # Evaluate on validation set
    model.eval()
    with torch.no_grad():
        y_pred_val = model(X_val)
        val_loss = criterion(y_pred_val, y_val)

    train_losses.append(loss.item())
    val_losses.append(val_loss.item())

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {loss.item():.4f}, Val Lo
```

```
Epoch 10/96, Train Loss: 0.0215, Val Loss: 0.0186
Epoch 20/96, Train Loss: 0.0124, Val Loss: 0.0123
Epoch 30/96, Train Loss: 0.0097, Val Loss: 0.0102
Epoch 40/96, Train Loss: 0.0082, Val Loss: 0.0097
Epoch 50/96, Train Loss: 0.0075, Val Loss: 0.0091
Epoch 10/96, Train Loss: 0.0215, Val Loss: 0.0186
Epoch 20/96, Train Loss: 0.0124, Val Loss: 0.0123
Epoch 30/96, Train Loss: 0.0097, Val Loss: 0.0102
Epoch 40/96, Train Loss: 0.0082, Val Loss: 0.0097
Epoch 50/96, Train Loss: 0.0075, Val Loss: 0.0091
Training Progress: 100%|██████████| 96/96 [00:00<00:00, 146.66it/s]
Epoch 10/96, Train Loss: 0.0215, Val Loss: 0.0186
Epoch 20/96, Train Loss: 0.0124, Val Loss: 0.0123
Epoch 30/96, Train Loss: 0.0097, Val Loss: 0.0102
Epoch 40/96, Train Loss: 0.0082, Val Loss: 0.0097
Epoch 50/96, Train Loss: 0.0075, Val Loss: 0.0091
Training Progress: 100%|██████████| 96/96 [00:00<00:00, 146.66it/s]
Epoch 60/96, Train Loss: 0.0070, Val Loss: 0.0084
Epoch 70/96, Train Loss: 0.0065, Val Loss: 0.0079
Epoch 80/96, Train Loss: 0.0062, Val Loss: 0.0075
Epoch 90/96, Train Loss: 0.0058, Val Loss: 0.0071
```

(C) Model Performance

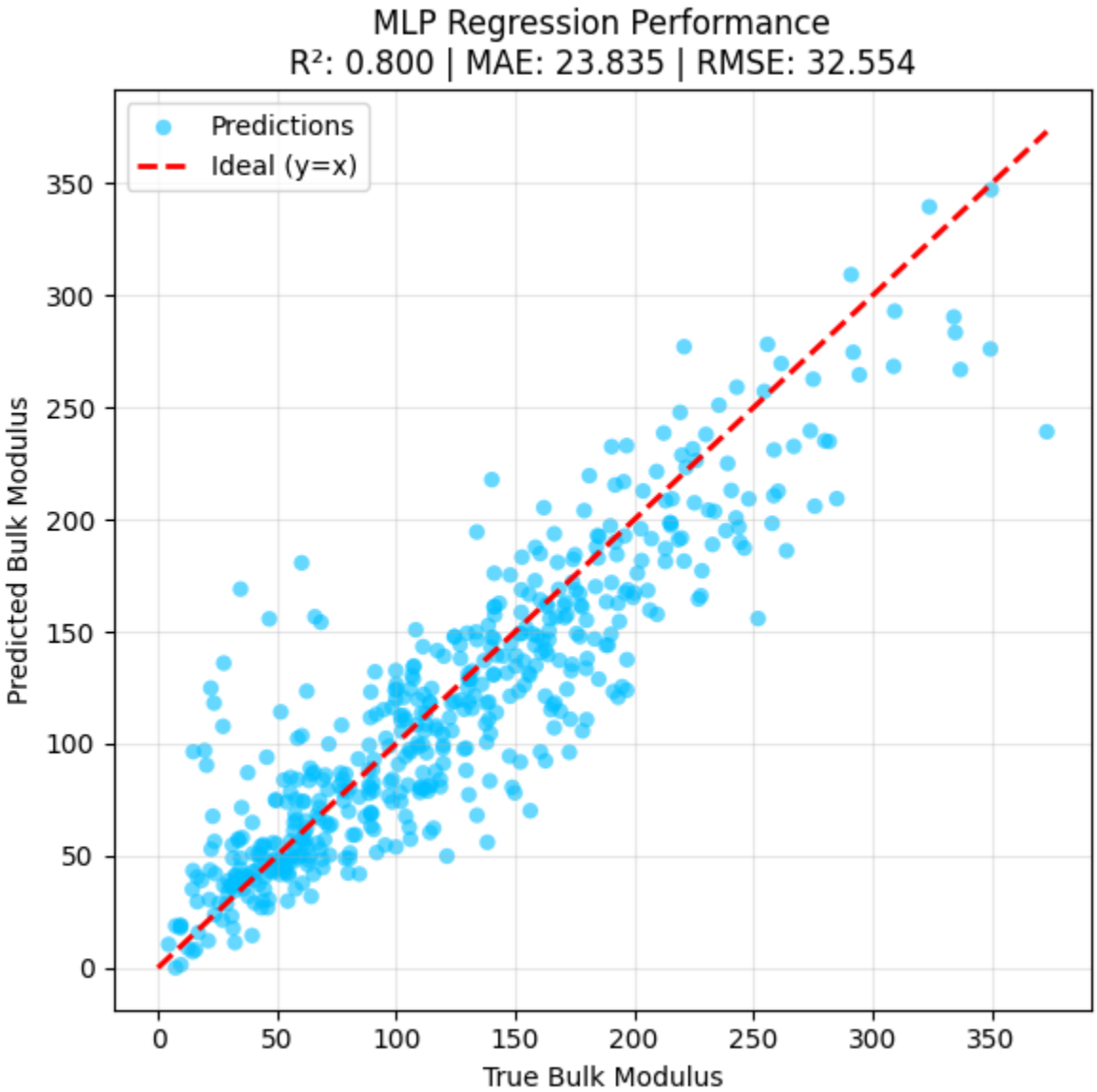
In [48]:
```
# Plot loss curves
plt.figure(figsize=(8, 5))
plt.plot(train_losses, label="Train Loss", color='blue')
plt.plot(val_losses, label="Testing Loss", color='orange')
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.title("Training Progress")
plt.minorticks_on()
plt.grid(which="major", linestyle="-", linewidth=0.8, alpha=1.0)
```

```
plt.grid(which="minor", linestyle=":", linewidth=0.6, alpha=0.75)
plt.show()
```



In [52]:
```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

# Predicted vs True values (validation set)
model.eval()
with torch.no_grad():
    y_pred_val = model(X_val)

# Convert to numpy
y_pred_scaled = y_pred_val.numpy()
y_true_scaled = y_val.numpy()

# Inverse-transform to original target units
y_pred_true_units = y_scaler.inverse_transform(y_pred_scaled)
y_true_true_units = y_scaler.inverse_transform(y_true_scaled)

# Scatter plot
plt.figure(figsize=(6, 6))
plt.scatter(y_true_true_units, y_pred_true_units, alpha=0.6, color="deepskyblue", e

# Ideal y=x line
line_min = min(y_true_true_units.min(), y_pred_true_units.min())
line_max = max(y_true_true_units.max(), y_pred_true_units.max())
plt.plot([line_min, line_max], [line_min, line_max], "--", color="red", linewidth=2

# Calculate performance metrics
r2 = r2_score(y_true_true_units, y_pred_true_units)
mae = mean_absolute_error(y_true_true_units, y_pred_true_units)
```

```
rmse = np.sqrt(mean_squared_error(y_true_true_units, y_pred_true_units))

plt.xlabel("True Bulk Modulus")
plt.ylabel("Predicted Bulk Modulus")
plt.title(f"MLP Regression Performance\nR²: {r2:.3f} | MAE: {mae:.3f} | RMSE: {rmse
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



6) Learning Strategies

(A) When I am try to learn something new there are several strategies I can use such as:

- Asking the expert to explain their reasoning step-by-step so that I can understand more fully
- Asking for a diversity of examples or use-cases of a concept I am learning

- Asking for analogies to areas that I already understand
- Ask about common misconceptions or areas that other people generally make mistakes in
- I could ask more than one expert for their opinion on a topic

This type of query strategy can be applied in machine learning:

- Providing a diverse set of data allows the model to generalize better and learn a more robust represenation
- Using multiple models to arrive at one conclusion (i.e. averaging predictions)
- When using LLMs it can often be beneficical to ask the model to explain its reasoning to allow to model to reach a more accurate result

(B) The paper already gives the right closed-loop structure, so active learning would basically be the experiment-selection engine inside their digital system. In their framework, the digital system uses data from printing + testing to choose the next formulations/process settings to run, and the physical system executes those experiments and feeds results back. The authors already point to the core pieces, Bayesian optimization for hitting target properties, optimal experimental design for choosing the most informative next experiments, and surrogate models (like ANNs) once enough data are collected.

A way to implement active leraning in their setup:

- Start with a small seed dataset (space-filling DOE) across composition + process settings.
- Train two models:
  - a property model (strength, modulus, density, etc.)
  - a feasibility model (will it print/test successfully or fail/jam)
- Use active learning to pick the next experiments:
  - exploit: Bayesian optimization to move toward target properties
  - explore: uncertainty-based selection to improve the model where it is weak
- Run the selected batch in the physical loop

(C) In practice, you would use the mycelium printing system as the experimental engine, then use the Lee-style digital layer to decide what to print next, nutrient level, nozzle size, line spacing, geometry, and growth/healing conditions.

This paper is especially good for this because it already shows clear tunable inputs and measurable outputs such as growth mode, gap bridging, self-healing, mechanical performance, and robotic skin behavior.

These are some ideas on how to apply machine learning:

- surrogate models to predict properties/performance
- active learning/Bayesian optimization to pick the next best prints
- computer vision to track growth, healing, and print defects over time
- interpretable ML to turn results into actual design rules, not just predictions

References

- Class textbook
- Lecture slides
- "NN example.ipynb" from Canvas
- GPT-5.3-Codex to review implementation of pyTorch code